

A Full Per-Token Trace of _s2_closer

every s-op as a row, every token as a column —
 HI = the boy that chases the dog swims

Generated by _gen_closer_handout.py; all values asserted equal to the module and grammar_oracle.py.

1 Stage 1 — encoding and part of speech

Each column is a token *position* (note *the* appears at both 0 and 4). Every row is one s-op: a length-7 array holding one value per position. Lexical-class codes: 0=D, 1=N_sing, 2=N_prop, 3=Adj, 4=Adv, 5=THAT, 6=V, 7=OTHER. Verb-role codes: 0=non-verb, 1=intransitive, 2=+object DP, 3=+clause, 4=object-gap.

s-op	the ₀	boy ₁	that ₂	chases ₃	the ₄	dog ₅	swims ₆
<i>The fixed word→id lookup _W2I (a token embedding). Each surface word becomes its unique integer id; repeated words (the) get the same id.</i>							
hi_ids	0	5	53	33	0	3	45
<i>The position s-op [0,1,...,n-1] from indices. The whole program's output is idx + disp, so this is the identity we perturb.</i>							
idx	0	1	2	3	4	5	6
<i>A second embedding tok_map(hi_ids, lexclass): id→coarse POS class. Shown by name; the underlying values are the integer codes above.</i>							
lc	D	N	THAT	V	D	N	V
<i>Pure lexical flag: 1 iff the word is in V_intrans. Only swims qualifies. Used later to tell an intransitive verb from an object-gap verb.</i>							
is_intr	0	0	0	0	0	0	1
<i>Backward neighbour read .read_off(lc,-1) = lc[i-1] (one attention head keyed on relative offset). -1 = fell off the left edge.</i>							
prev_lc	—	D	N	THAT	V	D	N
<i>lc[i-2]. Feeds the 'adjective-core' relative-clause test far below (has_adj_core).</i>							
prev2_lc	—	—	D	N	THAT	V	D
<i>Forward read lc[i+1]. This is the read that relies on the 'HI block is in the past' convention (it looks at a later position, legal only because the whole HI block precedes the output region).</i>							
next_lc	N	THAT	V	D	N	V	—
<i>lc[i+2], used to skip one optional post-verb adverb.</i>							
next2_lc	THAT	V	D	N	V	—	—
<i>(lc = THAT). Marks the clause openers — the only visible bracket-starts in the string. Fires only at position 2.</i>							
isthat	0	0	1	0	0	0	0
<i>isthat & (prev_lc = N): a that right after a common noun is a relative complementizer. Here boy(1) precedes that(2), so this fires.</i>							
is_crel	0	0	1	0	0	0	0
<i>isthat & ¬is_crel: the sentential that. Empty here — our only that is relative.</i>							
is_c	0	0	0	0	0	0	0
<i>(lc = V). True at chases(3) and swims(6).</i>							
isverb	0	0	0	1	0	0	1
<i>(lc = Adv). Empty here (no adverb).</i>							
isadv	0	0	0	0	0	0	0
<i>'next non-adverb class': where(next_lc=Adv, next2_lc, next_lc). Lets a verb see its complement's first token even across an adverb. With no adverb it equals next_lc.</i>							
nn_lc	N	THAT	V	D	N	V	—
<i>The verb role, from nn_lc: chases(3) sees the=D ⇒ 2 (transitive); swims(6) is lexically intransitive ⇒ 1. Roles 2/3 head a flip VP; 1/4 do not.</i>							
role	0	0	0	2	0	0	1

2 Stage 2 scaffolding — one-shot local span pieces

These are computed once, *before* the loop; each is a fixed-offset lookahead, no iteration.

s-op	the ₀	boy ₁	that ₂	chases ₃	the ₄	dog ₅	swims ₆
<i>Verb head-block end: i+1 for a verb + adverb, else i. No adverbs here, so it equals idx. This is the ‘head’ side of a VP flip.</i>							
hb_end	0	1	2	3	4	5	6
<i>Where a DP ends ignoring relative clauses: D (Adj) N. the(0)→boy(1); the(4)→dog(5). Note it does not yet know about the relative clause on boy.</i>							
dp_end_flat	1	1	2	3	5	5	6
<i>dp_end_flat + 1: the slot just past a DP’s noun, where a relative that would sit. For the(0) this is 2.</i>							
rc_that_pos	2	2	3	4	6	6	7
<i>index_select(lc, rc_that_pos): the class actually sitting at that slot (one cross-position lookahead). At position 0 it reads lc[2] = THAT.</i>							
lc_at_rc	THAT	THAT	V	D	V	V	–
<i>(lc = D) & (lc_at_rc = THAT): this DP carries a relative clause. True at the(0) — ‘the boy that...’.</i>							
dp_has_rc	1	0	0	0	0	0	0

3 Stage 2 relaxation — the loop, all 10 rounds

Four END pointers are seeded to a lower bound (‘each ends at itself’) and refined by 10 sweeps. A **red** cell changed from the row above; once a table stops turning red it has reached its fixed point. Only the openers’/verbs’ columns carry meaning — other columns hold the harmless **idx** filler.

dpfull. DP end including a trailing relative clause. Watch column *the*₀: at **init** it is the flat 1; it cannot finish until the relative clause’s end is known, so it lags one sweep behind **rce** and only settles at **R2**.

round	the ₀	boy ₁	that ₂	chases ₃	the ₄	dog ₅	swims ₆
init	1	1	2	3	5	5	6
R1	2	1	2	3	5	5	6
R2	5	1	2	3	5	5	6
R3	5	1	2	3	5	5	6
R4	5	1	2	3	5	5	6
R5	5	1	2	3	5	5	6
R6	5	1	2	3	5	5	6
R7	5	1	2	3	5	5	6
R8	5	1	2	3	5	5	6
R9	5	1	2	3	5	5	6
R10	5	1	2	3	5	5	6

vpe. VP end (verb + complement). *chases*₃ jumps to its object DP’s end (5) in R1; *swims*₆ is intransitive so it never moves off 6.

round	the ₀	boy ₁	that ₂	chases ₃	the ₄	dog ₅	swims ₆
init	0	1	2	3	4	5	6
R1	0	1	2	5	4	5	6
R2	0	1	2	5	4	5	6
R3	0	1	2	5	4	5	6
R4	0	1	2	5	4	5	6
R5	0	1	2	5	4	5	6
R6	0	1	2	5	4	5	6
R7	0	1	2	5	4	5	6
R8	0	1	2	5	4	5	6
R9	0	1	2	5	4	5	6
R10	0	1	2	5	4	5	6

rce. CP_{rel} end. *that*₂ is a subject-gap relative, so its end = the VP right after it (**vpe** at position 3), reaching 5 in R1.

round	the ₀	boy ₁	that ₂	chases ₃	the ₄	dog ₅	swims ₆
init	0	1	2	3	4	5	6
R1	0	1	5	3	4	5	6
R2	0	1	5	3	4	5	6
R3	0	1	5	3	4	5	6
R4	0	1	5	3	4	5	6
R5	0	1	5	3	4	5	6
R6	0	1	5	3	4	5	6
R7	0	1	5	3	4	5	6
R8	0	1	5	3	4	5	6
R9	0	1	5	3	4	5	6
R10	0	1	5	3	4	5	6

cpe. CP_sent end. Never active here (no sentential *that*), so it stays at the **idx** seed every round — a useful negative example.

round	the ₀	boy ₁	that ₂	chases ₃	the ₄	dog ₅	swims ₆
init	0	1	2	3	4	5	6
R1	0	1	2	3	4	5	6
R2	0	1	2	3	4	5	6
R3	0	1	2	3	4	5	6
R4	0	1	2	3	4	5	6
R5	0	1	2	3	4	5	6
R6	0	1	2	3	4	5	6
R7	0	1	2	3	4	5	6
R8	0	1	2	3	4	5	6
R9	0	1	2	3	4	5	6
R10	0	1	2	3	4	5	6

After R2 nothing changes: the depth-1 sentence has reached the fixed point with rounds 3–10 merely confirming it. A depth- d sentence would keep turning red through about round $d+1$.

4 Stage 2 output — clause ends and depth

Collapse the pointers into one clause-end per *that*, then count.

s-op	the ₀	boy ₁	that ₂	chases ₃	the ₄	dog ₅	swims ₆
<i>Clause end e(q): rce for a relative that, cpe for a sentential one, else idx. Here e(2) = 5: ‘that chases the dog’ closes on dog(5).</i>							
e	0	1	5	3	4	5	6
<i>where(isthat, e, -1): mask non-openers to -1 so they carry no interval into the count.</i>							
end_that	–	–	5	–	–	–	–
<i>The stabbing count $\#\{that\ q \leq i: e(q) \geq i\}$ (one attention head). Positions 2–5 are inside the one relative clause $\Rightarrow$ depth 1; the rest 0.</i>							
depth	0	0	1	1	1	1	0

5 Stage 2 output — displacement, four flip-typed pieces

Each piece is a (weighted) stabbing sum over enclosing flips of one type, plus a local head bonus. They add to **disp**.

s-op	the ₀	boy ₁	that ₂	chases ₃	the ₄	dog ₅	swims ₆
<i>Adverb flip (verb+1, adverb-1). All zero — no adverb.</i>							
d_adv	0	0	0	0	0	0	0
<i>e - idx + 1: the size of the clause an opener spans (only meaningful at that₂: 5 - 2 + 1 = 4).</i>							
span_clause	1	1	4	1	1	1	1

s-op	the ₀	boy ₁	that ₂	chases ₃	the ₄	dog ₅	swims ₆
$-depth + (\text{that? } span_clause : 0)$. Every token in a clause body gets -1 per enclosing that; the that itself gets $+ clause $. (Here no sentential that, so the clause body -1 's come via the relative-clause piece below instead — d_cp is 0 because is_c is empty and $depth$'s sign is carried by d_np_rel .)							
d_cp	0	0	3	—	—	—	0
$e[i+1]$: the clause end of a that sitting at $i+1$. Read by a noun to size the relative clause it is about to be modified by.							
e_at_p1	1	5	3	4	5	6	0
A common noun immediately before a relative that gets $+ CP_rel $. $boy(1)$: $e[2] - 2 + 1 = 4$. This is the noun jumping over its relative clause.							
$core_noun_bonus$	0	4	0	0	0	0	0
Same bonus for the adjective of an Adj N core carrying a relative clause. Zero here (no adjective).							
$core_adj_bonus$	0	0	0	0	0	0	0
$core_noun_bonus + core_adj_bonus$: the $+ RC $ for the modified noun. $boy(1) = +4$.							
$d_np_rel_core$	0	4	0	0	0	0	0
$(prev2_lc = Adj)$: does this relative clause's core noun carry an adjective (so $ core = 2$)? No.							
has_adj_core	0	0	0	0	0	0	0
Relative-clause ends with weight-1 core (mask others to -1), fed to a stabbing head.							
$crel_end_w1$	—	—	5	—	—	—	—
$-(stab(w1) + 2 \cdot stab(w2))$: every token inside a relative clause loses $ core $. Positions 2–5 each -1 .							
$d_np_rel_inside$	0	0	—	—	—	—	0
$d_np_rel_core + d_np_rel_inside$: the full NP-relative contribution. $boy+4$; the clause body -1 each.							
d_np_rel	0	4	—	—	—	—	0
$vpe - hb_end$: a VP-flip verb's complement size. $chases(3)$: $5 - 3 = 2$ ('the dog').							
$comp_len$	0	0	0	2	0	0	0
$(role = 2) \vee (role = 3)$: verbs that actually reverse with a complement. $chases(3)$ only.							
is_vpflip	0	0	0	1	0	0	0
VP-flip head (verb, and its adverb) gets $+ comp $. $chases(3) = +2$.							
d_vp_head	0	0	0	2	0	0	0
Marks a complement-start: the token whose previous token is a no-adverb VP-flip verb. $the(4)$, right after $chases$.							
$w1_here$	0	0	0	0	1	0	0
That complement's end (from $vpe[i-1]$), fed to a stabbing head.							
cs_end_w1	—	—	—	—	5	—	—
$-(stab(w1) + 2 \cdot stab(w2))$: complement tokens lose $ head $ per enclosing VP flip. $the(4), dog(5)$ each -1 .							
d_vp_comp	0	0	0	0	—	—	0
$d_vp_head + d_vp_comp$: full transitive/clausal-VP contribution.							
d_vp	0	0	0	2	—	—	0

6 Stage 3 — assemble and gather

Sum the pieces to a target slot per token, invert the permutation, gather.

s-op	the ₀	boy ₁	that ₂	chases ₃	the ₄	dog ₅	swims ₆
$d_adv + d_cp + d_np_rel + d_vp$. This equals the oracle's displacement exactly.							
$disp$	0	4	2	0	-3	-3	0
$idx + disp$: the HF slot each HI token targets. It is a genuine permutation of $0 \dots 6$.							
hf_pos	0	5	4	3	1	2	6
$kqv(hf_pos, idx, idx, equals)$: the inverse — for slot j , the source index i with $hf_pos[i] = j$.							
src	0	4	5	3	2	1	6
$index_select(hi_ids, src)$: gather the token id sitting at each source index into its slot.							
out_ids	0	0	3	33	53	5	45

Decoding `out_ids` back through `_I2W` gives the head-final string:

```
HF = the the dog chases that boy swims
```